

C Control Flow and Problem Solving

Focused revision sheet for the RAG item: if, for, while, factorial, numerical approximation of e, and compile/link/run workflow.

Description: What You Need To Know

1. Conditions and truth

In C, conditions use integer-style truth. Zero is false. Any non-zero value is true. Comparisons produce an integer result that can be used by `if`, `while`, and `for`.

```
int n = 7;

if (n > 0) {
    printf("positive\n");
} else if (n == 0) {
    printf("zero\n");
} else {
    printf("negative\n");
}
```

The comparison `n > 0` is tested first. Only one branch of this chain runs.

2. for loops

A for loop is useful when you know the counting pattern. The three parts are initialisation, condition, and update.

```
for (int i = 0; i < 5; i++) {
    printf("%d\n", i);
}
```

- `int i = 0` runs once before the loop starts.
- `i < 5` is checked before each iteration.
- `i++` runs after each loop body.
- The printed values are 0 through 4, not 5.

3. while loops

A while loop is useful when the number of iterations depends on a condition that changes during the loop.

```
int n = 10;

while (n > 1) {
    printf("%d\n", n);
    n = n / 2;
}
```

Make sure the loop body changes something involved in the condition. Otherwise the loop may never finish.

4. Factorial pattern

Factorial multiplies the integers from 1 to `n`. Start the accumulator at 1, not 0, because multiplying by zero destroys the result.

```
int factorial(int n) {
    int result = 1;

    for (int i = 2; i <= n; i++) {
        result *= i;
    }

    return result;
}
```

For `n == 5`, the updates are `1 * 2 * 3 * 4 * 5`, so the result is 120.

5. Approximating e

The constant `e` can be approximated using the series `1 + 1/1! + 1/2! + 1/3! + ...`. This is a good loop exercise because each term depends on the previous factorial.

```
double approximate_e(int terms) {
    double sum = 1.0;
    double factorial = 1.0;

    for (int n = 1; n < terms; n++) {
        factorial *= n;
        sum += 1.0 / factorial;
    }
}
```

```

    }
    return sum;
}

```

Use `1.0`, not `1`, when you need floating-point division.

6. Early returns and input checks

Small C programs are easier to reason about when invalid cases are rejected early. Check preconditions before doing the main work.

```

int safe_factorial(int n, int *out) {
    if (n < 0) {
        return 0;
    }

    int result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }

    *out = result;
    return 1;
}

```

Here the return value is a success flag. The output value is written only when the input is valid.

7. Compile, link, and run

For a single-file C program, one command can compile and link. For multiple files, you can compile to object files first and then link them.

```

gcc -Wall -Wextra -std=c11 -o app main.c
./app

gcc -Wall -Wextra -std=c11 -c main.c
gcc -Wall -Wextra -std=c11 -c maths.c
gcc -o app main.o maths.o
./app

```

- `-c` compiles a source file to an object file without linking.
- The final link command combines object files into an executable.
- Warnings should be read as useful feedback, not ignored.

Practice Questions

- 1 Trace the output of a loop that runs for `(int i = 0; i < 4; i++)`.
- 2 Write an `if` statement that prints whether an integer is negative, zero, or positive.
- 3 Write a `while` loop that divides a value by two until it reaches zero.
- 4 Write a factorial function for non-negative integers using a loop and an accumulator.
- 5 Explain why a factorial accumulator starts at `1` instead of `0`.
- 6 Write a loop that approximates `e` using terms terms.
- 7 Given a function that cannot handle negative input, show the early-return check.
- 8 Write the compile and run commands for `main.c` using `gcc`.
- 9 For a program split across `main.c` and `maths.c`, write the commands that create `main.o` and `maths.o`, then link them into `app`.
- 10 Explain why `1.0 / factorial(n)` is used in the `e` approximation instead of `1 / factorial(n)`.

Mark Scheme

Use this after attempting the questions. Strong answers should trace loop state, not just quote syntax.

- 1 The loop prints 0, 1, 2, and 3. It stops when `i` becomes 4.
- 2 Expected: use `if (n < 0), else if (n == 0), and else`.
- 3 Expected shape: initialise `n`, test `n != 0` or `n > 0`, update `n = n / 2` in the body.
- 4 Expected: `int result = 1;`, loop from 2 to `n`, multiply into `result`, then return it.
- 5 Multiplication uses 1 as the neutral starting value. Starting at 0 would keep the result at zero.
- 6 Expected: keep a running factorial and add `1.0 / factorial` each iteration.
- 7 Expected shape: `if (n < 0) return 0;` before the main calculation.
- 8 Expected single-file command: `gcc -Wall -Wextra -std=c11 -o app main.c`, then `./app`.
- 9 Expected: `gcc -Wall -Wextra -std=c11 -c main.c`, `gcc -Wall -Wextra -std=c11 -c maths.c`, then `gcc -o app main.o maths.o`.
- 10 Using `1.0` forces floating-point division. With integer division, terms such as `1 / 2` and `1 / 6` become 0.

Common Mistakes

- Using `i <= n` when the valid range should stop before `n`.
- Forgetting to update the variable used in a `while` condition.
- Starting a product accumulator at zero.
- Using integer division where floating-point division is needed.
- Ignoring compiler warnings during the compile-link-run cycle.